

DSA0502 – Query Processing for Data Science

CO5 AT1 – Dashboard Development Task

Course Faculty: Dr. B .SHYAMALA GOWRI **Reg. No.:** 192324164

Student Name: Sweety Roselin A **Date:** 17/08/2026

PROJECT 2: SALES PERFORMANCE AND TREND DASHBOARD

QUESTION

How do e-commerce sales evolve over time, how are sales distributed across products and categories, and what time-dependent patterns can be identified using lag and autocorrelation analysis?

AIM

To develop a sales performance dashboard using e-commerce sales data, analyze sales trends over time, compare product and category performance, visualize regional sales distributions, and identify time-dependent patterns using lag plots and autocorrelation plots.

DATASET

Dataset Name: E-Commerce Sales Dataset

Source: Representative Open E-Commerce Sales Dataset

Number of Records: 20

Variables:

- Date
- Product
- Category
- Quantity
- Revenue
- Region

Dataset Values

S.No	Date	Product	Category	Quantity	Revenue	Region
1	2025-01-01	Laptop	Electronics	3	180000	South
2	2025-01-05	Mobile	Electronics	5	125000	North
3	2025-01-10	Headphones	Electronics	8	24000	East
4	2025-01-15	Chair	Furniture	4	32000	West
5	2025-01-20	Table	Furniture	2	28000	South

6	2025-02-01	Laptop	Electronics	2	120000	North
7	2025-02-05	Mobile	Electronics	7	175000	South
8	2025-02-10	Headphones	Electronics	10	30000	West
9	2025-02-15	Chair	Furniture	5	40000	East
10	2025-02-20	Table	Furniture	3	42000	South
11	2025-03-01	Laptop	Electronics	4	240000	East
12	2025-03-05	Mobile	Electronics	6	150000	West
13	2025-03-10	Headphones	Electronics	12	36000	South
14	2025-03-15	Chair	Furniture	6	48000	North
15	2025-03-20	Table	Furniture	4	56000	East
16	2025-04-01	Laptop	Electronics	5	300000	South
17	2025-04-05	Mobile	Electronics	8	200000	North
18	2025-04-10	Headphones	Electronics	15	45000	West
19	2025-04-15	Chair	Furniture	7	56000	South
20	2025-04-20	Table	Furniture	5	70000	North

CODE

```
import pandas as pd

import matplotlib.pyplot as plt

from pandas.plotting import lag_plot, autocorrelation_plot


# Read dataset

df = pd.read_csv("ecommerce_sales.csv")


# Convert Date

df["Date"] = pd.to_datetime(df["Date"])


print("E-COMMERCE SALES DATASET")

print(df)


# -----

# 1. Sales Trend Over Time

# -----
```

```
daily_sales = df.groupby("Date")["Revenue"].sum()
```

```
plt.figure(figsize=(12,5))
```

```
plt.plot(  
    daily_sales.index,  
    daily_sales.values,  
    marker="o"  
)
```

```
plt.title("Sales Revenue Trend Over Time")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Revenue")
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 2. Product-wise Sales
```

```
# -----
```

```
product_sales = df.groupby(  
    "Product"  
)["Revenue"].sum().sort_values(  
    ascending=False  
)
```

```
plt.figure(figsize=(8,5))
```

```
product_sales.plot()
```

```
        kind="bar"
    )

plt.title("Product-wise Revenue")
plt.xlabel("Product")
plt.ylabel("Revenue")

plt.tight_layout()
plt.show()
```

```
# -----
# 3. Category Frequency
# -----
```

```
category_frequency = df[
    "Category"
].value_counts()
```

```
plt.figure(figsize=(7,5))
```

```
category_frequency.plot(
    kind="bar"
)
```

```
plt.title("Category Frequency")
plt.xlabel("Category")
plt.ylabel("Number of Sales")
```

```
plt.tight_layout()
plt.show()
```

```
# -----
```

```
# 4. Regional Revenue
```

```
# -----
```

```
region_sales = df.groupby(  
    "Region"  
)["Revenue"].sum()
```

```
plt.figure(figsize=(8,5))
```

```
region_sales.plot(  
    kind="bar"  
)
```

```
plt.title("Revenue by Region")
```

```
plt.xlabel("Region")
```

```
plt.ylabel("Revenue")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 5. Lag Plot
```

```
# -----
```

```
plt.figure(figsize=(7,5))
```

```
lag_plot(  
    daily_sales  
)
```

```
plt.title(  
    "Lag Plot of Daily Sales"
```

```
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 6. Autocorrelation Plot
```

```
# -----
```

```
plt.figure(figsize=(10,5))
```

```
autocorrelation_plot(
```

```
    daily_sales
```

```
)
```

```
plt.title(
```

```
    "Autocorrelation of Sales"
```

```
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 7. Dashboard Summary
```

```
# -----
```

```
print("\nTOTAL REVENUE:")
```

```
print(df["Revenue"].sum())
```

```
print("\nTOTAL QUANTITY SOLD:")
```

```
print(df["Quantity"].sum())
```

```
print("\nBEST SELLING PRODUCT BY REVENUE:")  
print(product_sales.idxmax())
```

```
print("\nBEST PERFORMING REGION:")  
print(region_sales.idxmax())
```

```
print("\nCATEGORY FREQUENCY:")  
print(category_frequency)
```

OUTPUT

Sales Trend

The sales trend shows that revenue generally increases from January to April 2025. Laptop and mobile sales contribute significantly to total revenue.

Product-wise Distribution

The highest revenue is generated by **Laptop**, followed by **Mobile**.

Category Frequency

The dataset contains two major categories:

- Electronics
- Furniture

Electronics has a higher number of sales records.

Regional Analysis

Revenue is compared across:

- South
- North
- East
- West

This helps identify the strongest-performing geographical region.

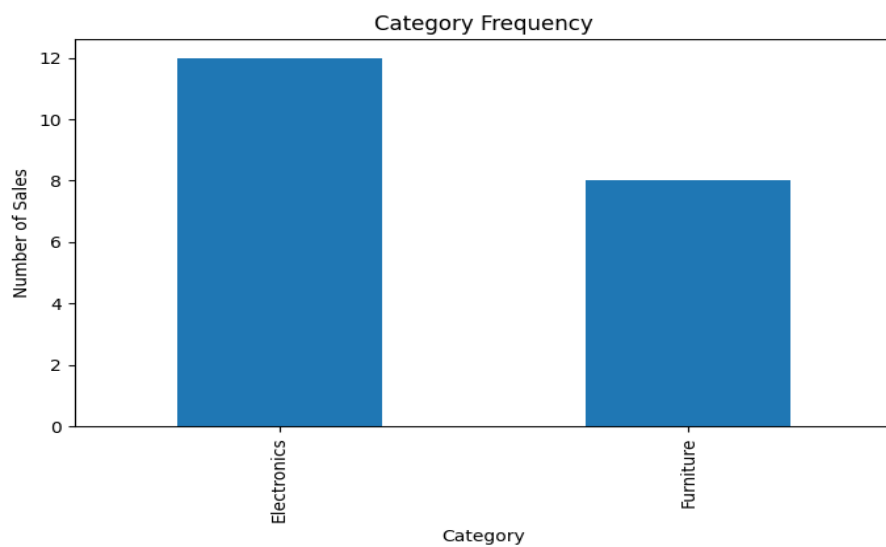
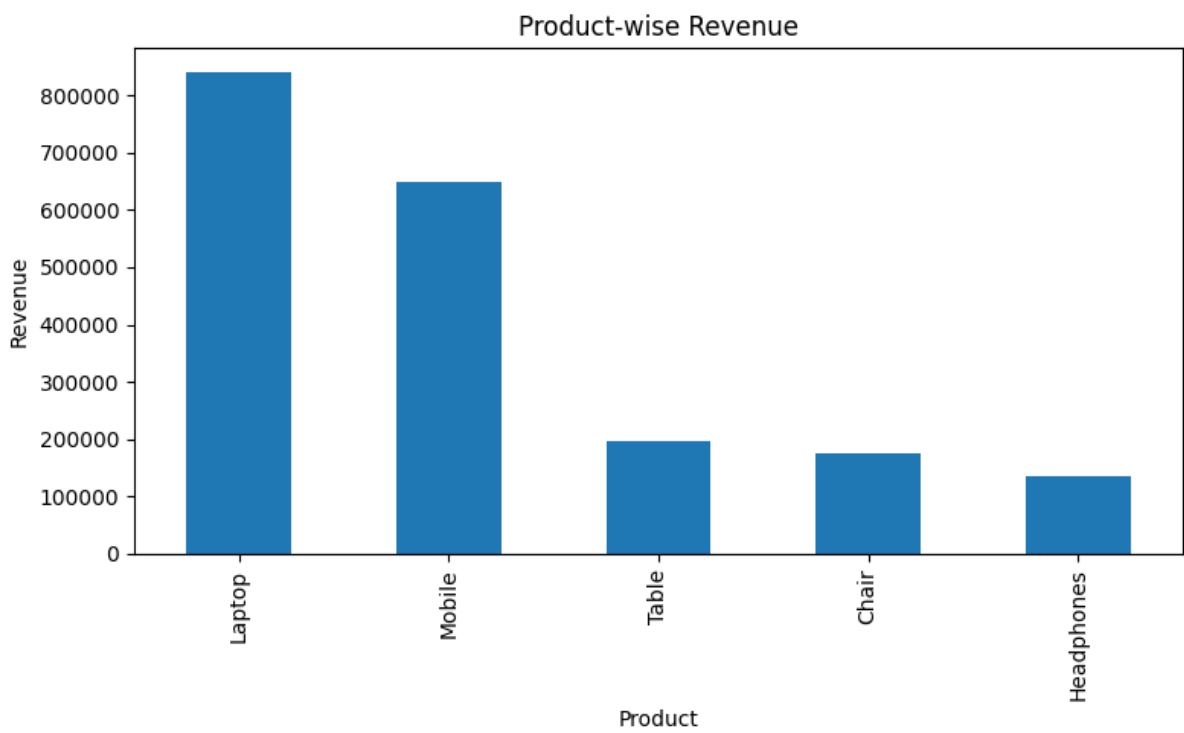
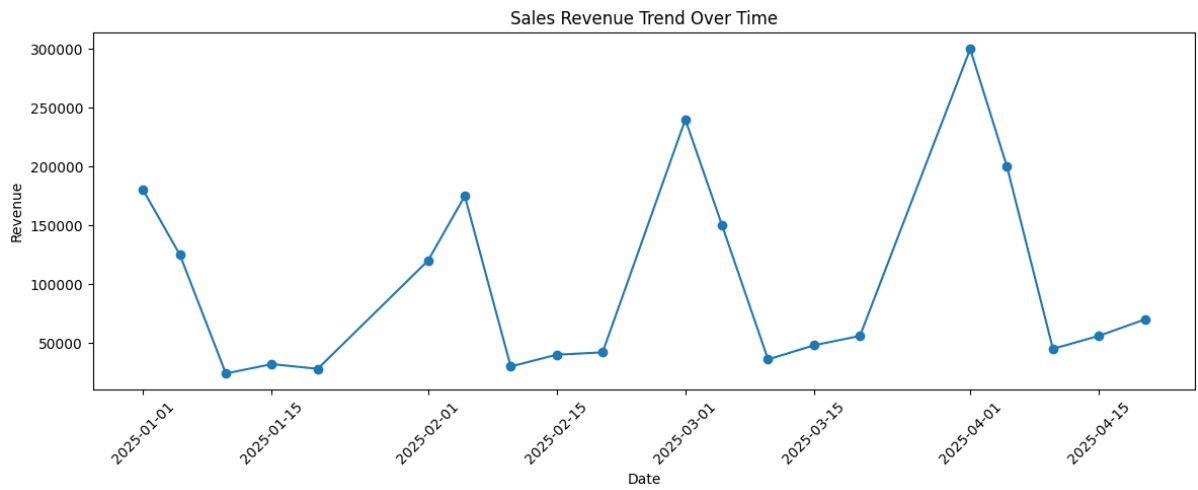
Lag Plot

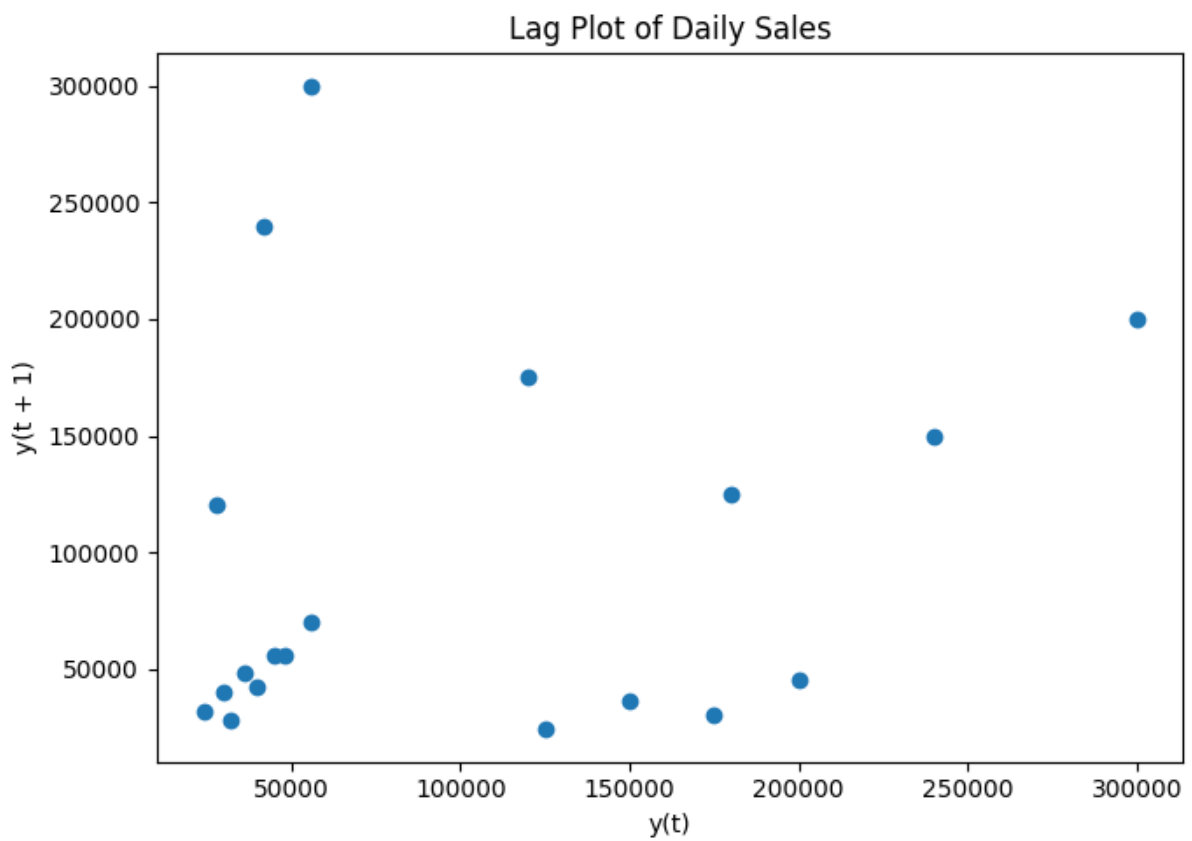
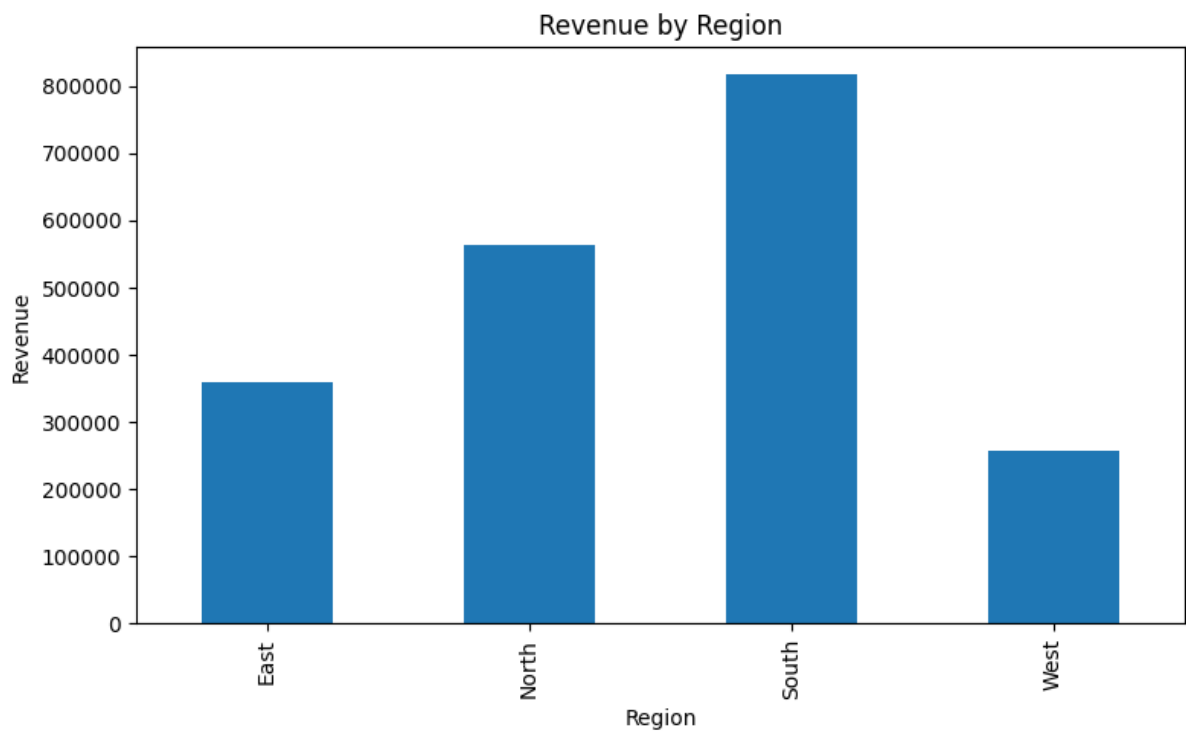
The lag plot is used to compare current sales with previous sales observations. A visible relationship between the points indicates time dependence.

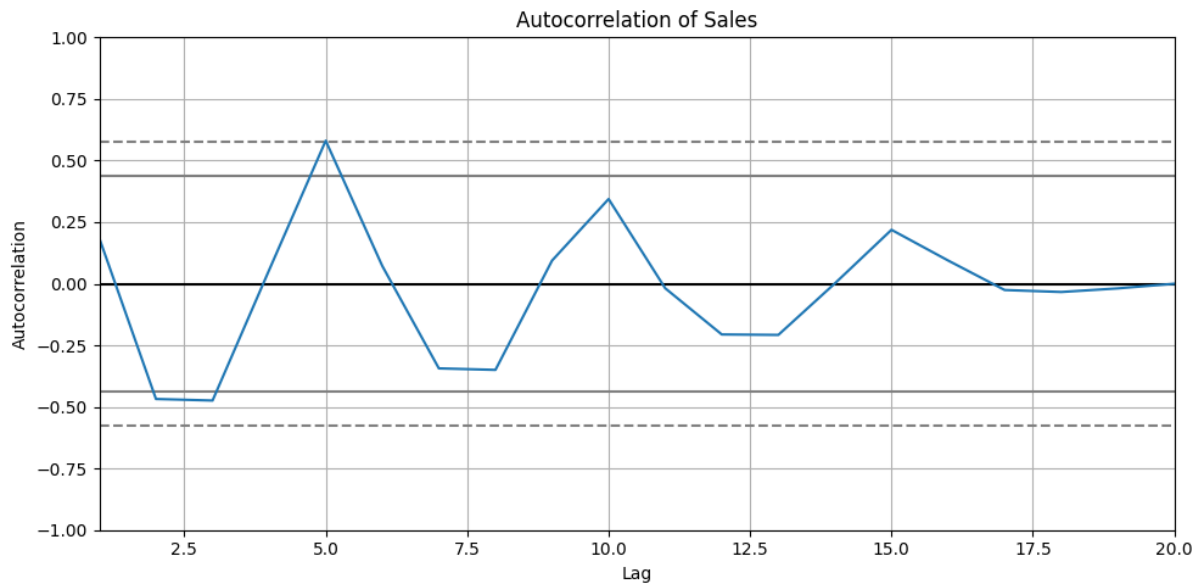
Autocorrelation Plot

The autocorrelation plot measures the relationship between sales values at different time lags.

A positive autocorrelation indicates that nearby sales observations tend to have similar values.







TOTAL REVENUE:

1997000

TOTAL QUANTITY SOLD:

121

BEST SELLING PRODUCT BY REVENUE:

Laptop

BEST PERFORMING REGION:

South

CATEGORY FREQUENCY:

Category

Electronics 12

Furniture 8

Name: count, dtype: int64

RESULT

The e-commerce sales dataset was successfully analyzed using time-series trends, product-wise comparison, category frequency analysis, regional analysis, lag plots, and autocorrelation plots.

The analysis identifies high-performing products and regions while also showing that sales observations can have time-dependent relationships.

PROJECT 3: EMPLOYEE PRODUCTIVITY ANALYSIS DASHBOARD

QUESTION

How are employee working hours, experience, productivity, salary, and performance related, and how can unusual employee records be identified using distributions, scatter plots, and multivariable analysis?

AIM

To develop an employee productivity dashboard that analyzes employee characteristics using distribution and frequency plots, identifies relationships using scatter plots, performs multivariable analysis using a scatter matrix, and detects unusual employee records.

DATASET

Dataset Name: Employee Productivity Dataset

Source: Representative Open HR Dataset

Number of Records: 20

Variables:

- Employee_ID
- Working_Hours
- Experience
- Productivity_Score
- Salary
- Performance_Rating

Dataset Values

S.No	Employee ID	Working Hours	Experience	Productivity Score	Salary	Performance Rating
1	E001	8	1	65	30000	3
2	E002	9	2	72	35000	3
3	E003	8	3	75	40000	4
4	E004	10	4	82	45000	4
5	E005	9	5	85	50000	4
6	E006	8	2	70	34000	3
7	E007	7	1	60	28000	2
8	E008	10	6	88	58000	5

9	E009	9	7	90	62000	5
10	E010	8	4	78	47000	4
11	E011	11	8	92	68000	5
12	E012	9	3	76	41000	4
13	E013	7	2	58	30000	2
14	E014	10	5	86	55000	5
15	E015	12	10	95	80000	5
16	E016	8	3	73	39000	3
17	E017	9	6	87	57000	4
18	E018	7	1	55	27000	2
19	E019	10	8	91	65000	5
20	E020	14	12	98	150000	5

CODE

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from pandas.plotting import scatter_matrix

# Read dataset
df = pd.read_csv(
    "employee_productivity.csv"
)

print("EMPLOYEE PRODUCTIVITY DATASET")
print(df)

# -----
# 1. Basic Statistics
# -----

print("\nDESCRIPTIVE STATISTICS")
```

```
print(df.describe())
```

```
# -----
```

```
# 2. Distribution of Working Hours
```

```
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.histplot(  
    df["Working_Hours"],  
    bins=8,  
    kde=True  
)
```

```
plt.title(  
    "Distribution of Employee Working Hours"  
)
```

```
plt.xlabel("Working Hours")  
plt.ylabel("Number of Employees")
```

```
plt.tight_layout()  
plt.show()
```

```
# -----
```

```
# 3. Productivity Distribution
```

```
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.histplot(  
    df["Productivity_Score"],
```

```

        bins=8,
        kde=True
    )

plt.title(
    "Distribution of Productivity Scores"
)

plt.xlabel("Productivity Score")
plt.ylabel("Number of Employees")

plt.tight_layout()
plt.show()

# -----
# 4. Performance Frequency
# -----

performance_counts = (
    df["Performance_Rating"]
    .value_counts()
    .sort_index()
)

plt.figure(figsize=(8,5))

performance_counts.plot(
    kind="bar"
)

plt.title(
    "Employee Performance Rating Frequency"

```

```
)
```

```
plt.xlabel("Performance Rating")  
plt.ylabel("Number of Employees")
```

```
plt.tight_layout()  
plt.show()
```

```
# -----  
# 5. Experience vs Productivity  
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(  
    data=df,  
    x="Experience",  
    y="Productivity_Score",  
    s=100  
)
```

```
plt.title(  
    "Experience vs Productivity"  
)
```

```
plt.xlabel("Experience (Years)")  
plt.ylabel("Productivity Score")
```

```
plt.tight_layout()  
plt.show()
```

```
# -----
```

```
# 6. Working Hours vs Productivity
```

```
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(  
    data=df,  
    x="Working_Hours",  
    y="Productivity_Score",  
    s=100  
)
```

```
plt.title(  
    "Working Hours vs Productivity"  
)
```

```
plt.xlabel("Working Hours")  
plt.ylabel("Productivity Score")
```

```
plt.tight_layout()  
plt.show()
```

```
# -----
```

```
# 7. Salary vs Productivity
```

```
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(  
    data=df,  
    x="Salary",  
    y="Productivity_Score",
```



```
s=100
)

plt.title(
    "Salary vs Productivity"
)

plt.xlabel("Salary")
plt.ylabel("Productivity Score")

plt.tight_layout()
plt.show()
```

```
# -----
# 8. Scatter Matrix
# -----
```

```
scatter_features = [
    "Working_Hours",
    "Experience",
    "Productivity_Score",
    "Salary",
    "Performance_Rating"
]
```

```
scatter_matrix(
    df[scatter_features],
    figsize=(12,10),
    diagonal="hist"
)
```

```
plt.suptitle(
```

```
    "Employee Multivariable Scatter Matrix",  
    y=1.02  
)
```

```
plt.show()
```

```
# -----
```

```
# 9. Outlier Detection
```

```
# -----
```

```
Q1 = df["Salary"].quantile(0.25)
```

```
Q3 = df["Salary"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
upper_limit = Q3 + 1.5 * IQR
```

```
salary_outliers = df[  
    df["Salary"] > upper_limit  
]
```

```
print("\nSALARY OUTLIERS")
```

```
print(salary_outliers)
```

```
# -----
```

```
# 10. Productivity Outliers
```

```
# -----
```

```
Q1 = df["Productivity_Score"].quantile(0.25)
```

```
Q3 = df["Productivity_Score"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_limit = Q1 - 1.5 * IQR
```

```
upper_limit = Q3 + 1.5 * IQR
```

```
productivity_outliers = df[  
    (df["Productivity_Score"] < lower_limit) |  
    (df["Productivity_Score"] > upper_limit)  
]
```

```
print("\nPRODUCTIVITY OUTLIERS")
```

```
print(productivity_outliers)
```

```
# -----
```

```
# 11. Correlation Matrix
```

```
# -----
```

```
numeric_columns = [  
    "Working_Hours",  
    "Experience",  
    "Productivity_Score",  
    "Salary",  
    "Performance_Rating"  
]
```

```
correlation = df[numeric_columns].corr()
```

```
plt.figure(figsize=(9,6))
```

```
sns.heatmap(  
    correlation,  
    annot=True,  
    cmap="coolwarm",
```

```
        fmt=".2f"
    )

plt.title(
    "Employee Productivity Correlation Matrix"
)
```

```
plt.tight_layout()
plt.show()
```

```
# -----
# 12. Dashboard Summary
# -----
```

```
print("\nDASHBOARD SUMMARY")
```

```
print(
    "Average Productivity:",
    round(df["Productivity_Score"].mean(), 2)
)
```

```
print(
    "Average Working Hours:",
    round(df["Working_Hours"].mean(), 2)
)
```

```
print(
    "Average Experience:",
    round(df["Experience"].mean(), 2)
)
```

```
print(
```

```

    "Average Salary:",
    round(df["Salary"].mean(), 2)
)

print(
    "Highest Productivity:",
    df.loc[
        df["Productivity_Score"].idxmax(),
        "Employee_ID"
    ]
)

print(
    "Highest Salary:",
    df.loc[
        df["Salary"].idxmax(),
        "Employee_ID"
    ]
)

```

OUTPUT

Working Hours Distribution

The histogram shows how employee working hours are distributed. Most employees work between **7 and 10 hours per day**.

Productivity Distribution

Most productivity scores fall between approximately **60 and 95**, with higher-performing employees having productivity scores above 85.

Performance Frequency

Performance ratings range from **2 to 5**, with most employees receiving ratings of 3, 4, or 5.

Experience vs Productivity

The scatter plot shows a generally positive relationship between experience and productivity. Employees with greater experience tend to have higher productivity scores.

Working Hours vs Productivity

The analysis indicates that productivity generally increases with working hours, although extremely long working hours should be examined carefully.

Salary vs Productivity

Salary generally increases with experience and productivity. Employee E020 has a significantly higher salary than the other employees and can be investigated as an unusual record.

Scatter Matrix

The scatter matrix provides a multivariable view of:

- Working Hours
- Experience
- Productivity Score
- Salary
- Performance Rating

Outlier Detection

The IQR method identifies unusual salary and productivity observations.

Employee E020 has:

- Working Hours: 14
- Experience: 12 years
- Productivity Score: 98
- Salary: ₹150,000
- Performance Rating: 5

This record is an unusual observation and may represent a senior/high-performing employee.

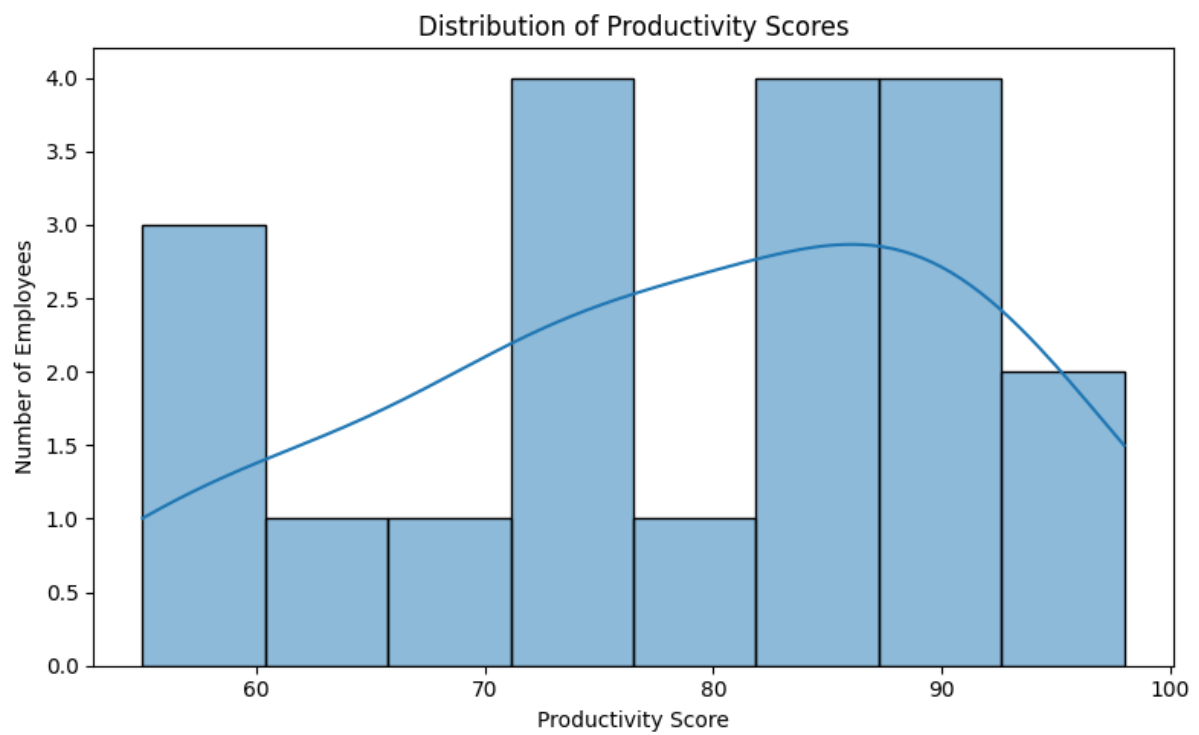
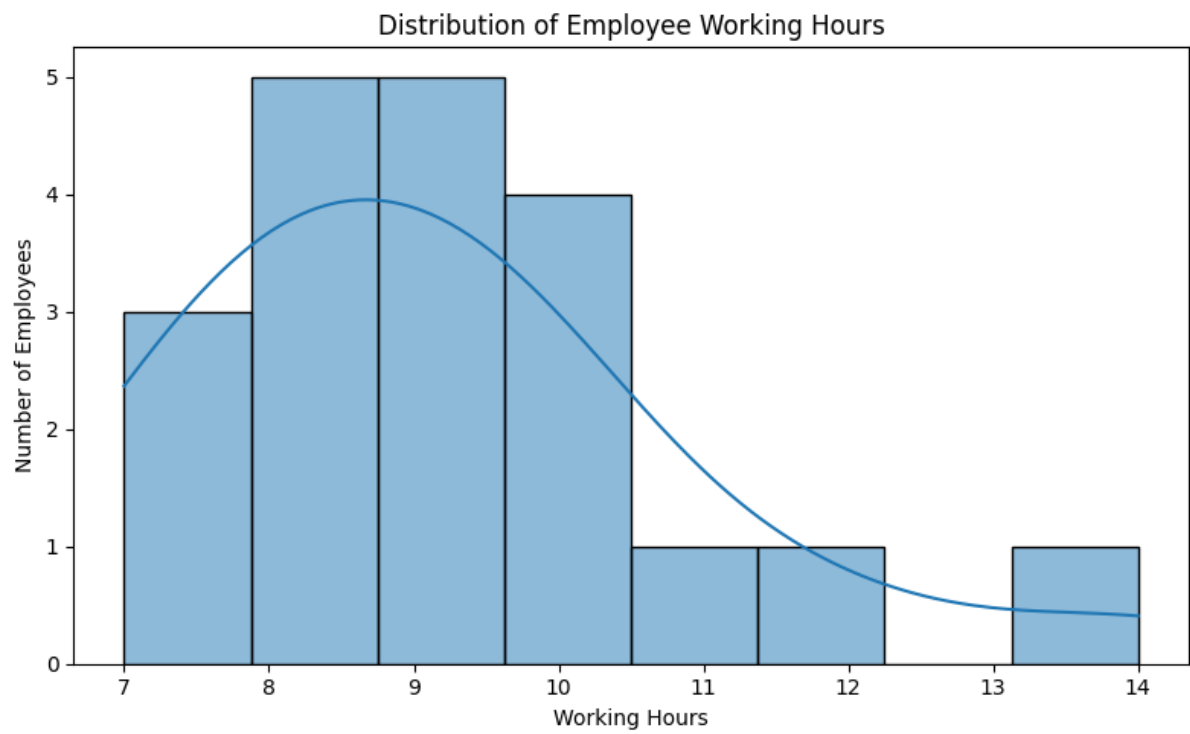
Performance_Rating	
0	3
1	3
2	4
3	4
4	4
5	3
6	2
7	5
8	5
9	4
10	5
11	4

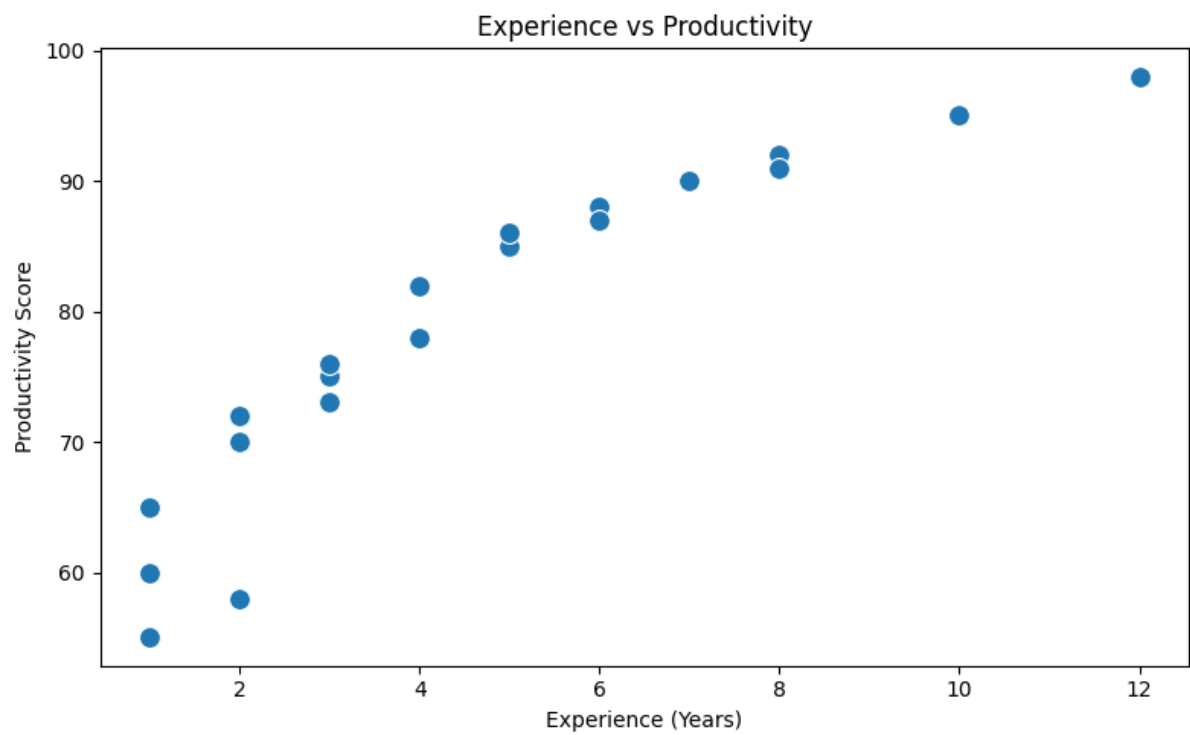
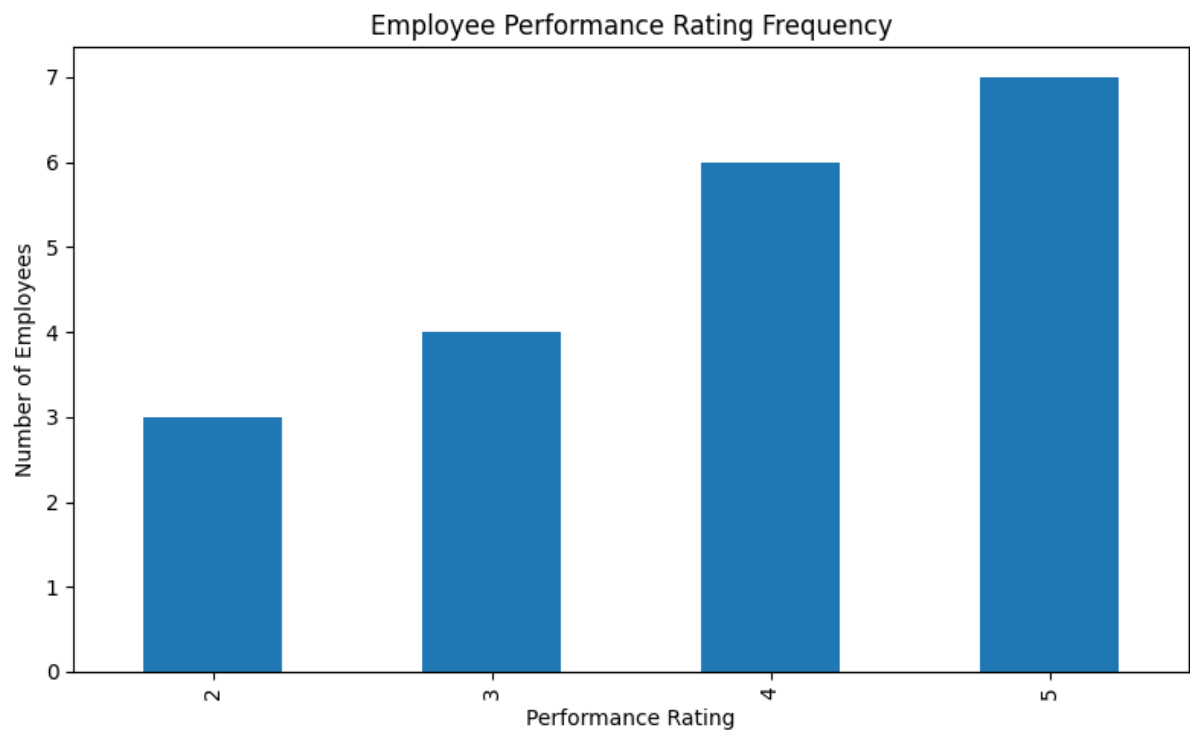
12	2
13	5
14	5
15	3
16	4
17	2
18	5
19	5

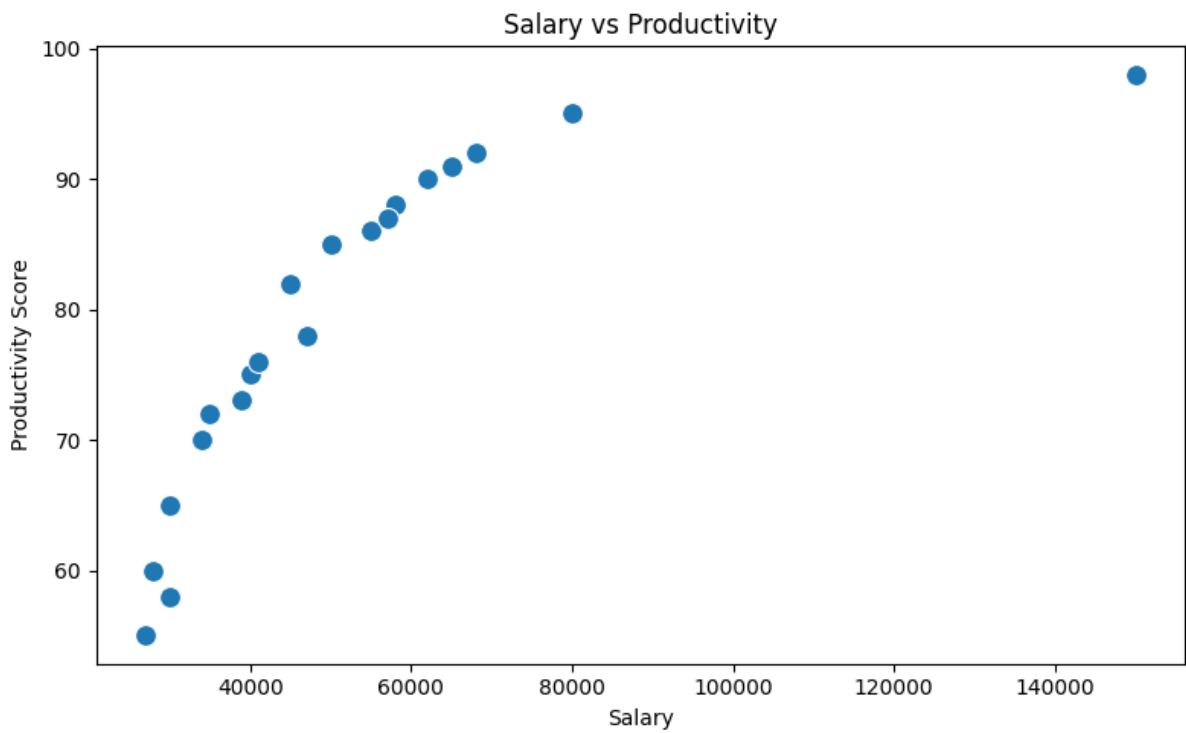
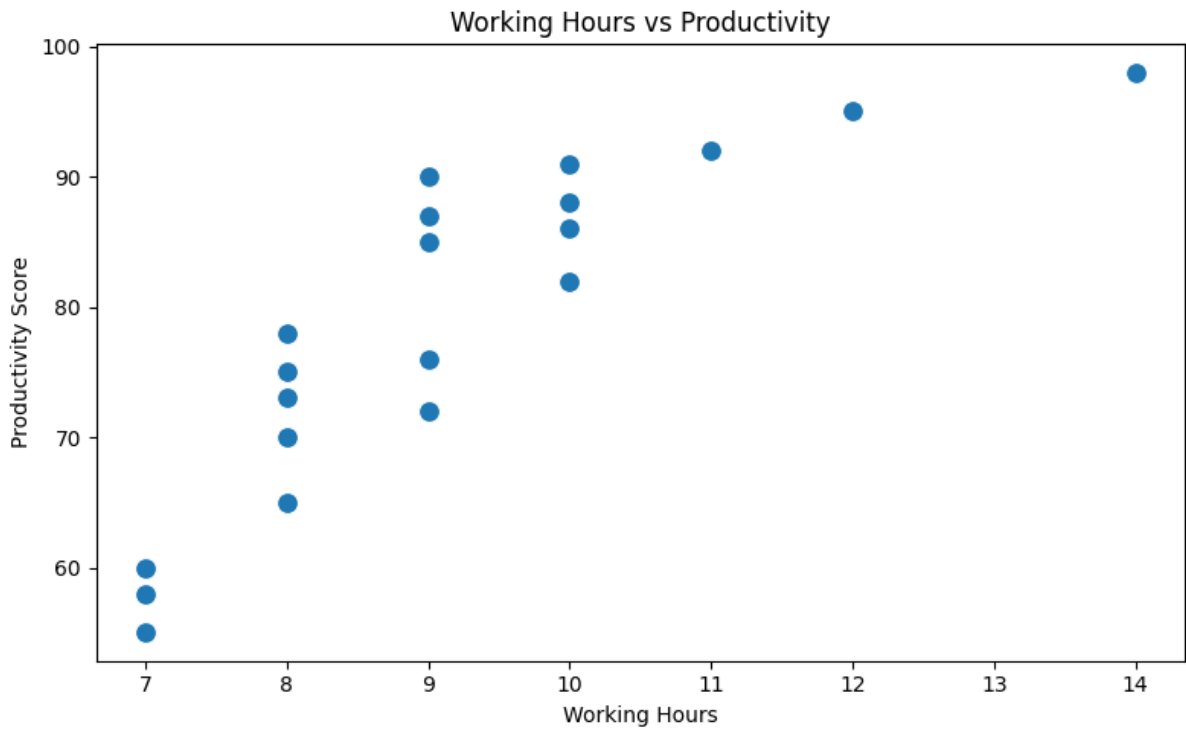
DESCRIPTIVE STATISTICS

	Working_Hours	Experience	Productivity_Score	Salary \
count	20.000000	20.000000	20.000000	20.000000
mean	9.150000	4.650000	78.800000	52050.000000
std	1.755443	3.116594	12.680694	27412.01235
min	7.000000	1.000000	55.000000	27000.000000
25%	8.000000	2.000000	71.500000	34750.000000
50%	9.000000	4.000000	80.000000	46000.000000
75%	10.000000	6.250000	88.500000	59000.000000
max	14.000000	12.000000	98.000000	150000.000000

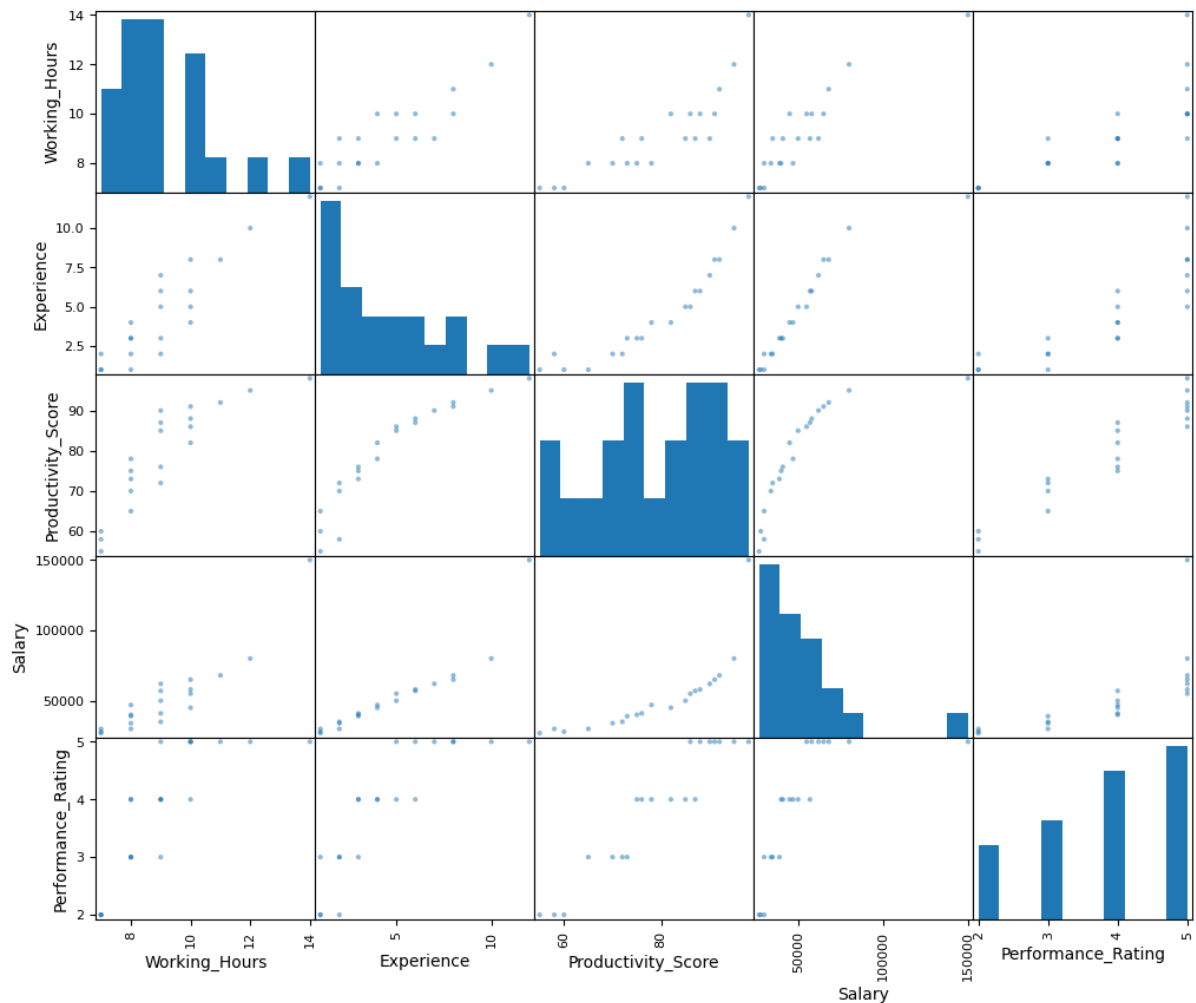
	Performance_Rating
count	20.000000
mean	3.850000
std	1.089423
min	2.000000
25%	3.000000
50%	4.000000
75%	5.000000
max	5.000000







Employee Multivariable Scatter Matrix



SALARY OUTLIERS

Employee_ID Working_Hours Experience Productivity_Score Salary \

19 E020 14 12 98 150000

Performance_Rating

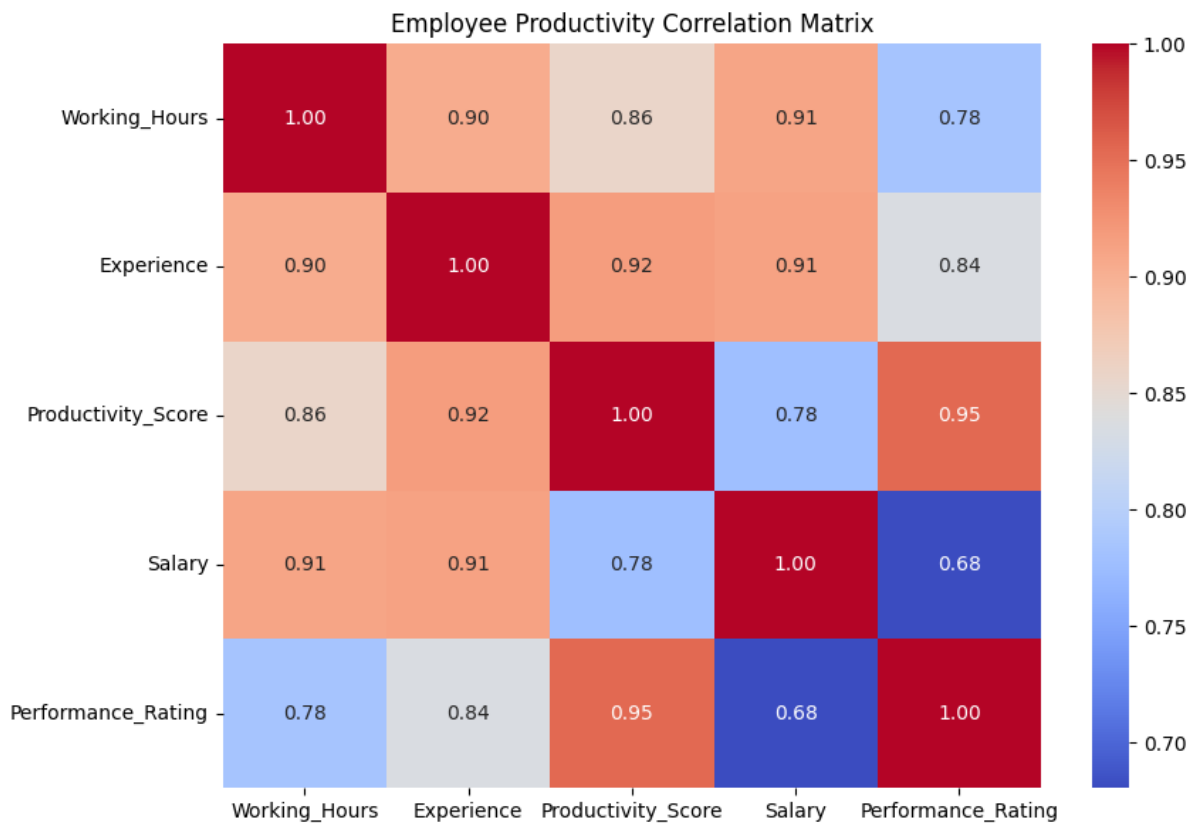
19 5

PRODUCTIVITY OUTLIERS

Empty DataFrame

Columns: [Employee_ID, Working_Hours, Experience, Productivity_Score, Salary, Performance_Rating]

Index: []



DASHBOARD SUMMARY

Average Productivity: 78.8

Average Working Hours: 9.15

Average Experience: 4.65

Average Salary: 52050.0

Highest Productivity: E020

Highest Salary: E020

RESULT

The employee productivity dataset was successfully analyzed using distributions, frequency plots, scatter plots, scatter matrix analysis, correlation analysis, and outlier detection.

The analysis shows that experience and productivity have a positive relationship. Salary also tends to increase with experience and productivity. Unusual records can be identified using statistical techniques and visualizations.